

Universidade do Minho

Escola de Engenharia

Mestrado em Engenharia de
Telecomunicações e Informática
Escola de Engenharia

Relatório técnico

GVR – Módulo de Gestão de Redes

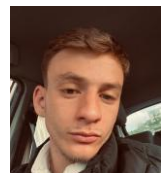
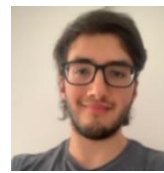
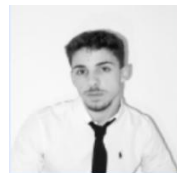
Sistema de Gestão de Tráfego Rodoviário baseado em SNMP

Grupo 3 constituído por:

Luís Pedro Cerqueira Baptista, PG61434

Bernardo Gualdino Salgado, PG59972

João Marcelo Mendes Teixeira, A101262



2025/2026

Índice

1. Introdução	5
2. Discussão das Estratégias e Opções Tomadas	5
2.1. Arquitetura Integrada e Memória Partilhada	5
2.2. Gestão de Concorrência e Consistência de Dados.....	6
2.3. Estratégia de Modelação da MIB e Controlo	6
2.4. Algoritmo de Decisão e Desacoplamento Temporal	6
2.5. Opções Tecnológicas	7
3. Enquadramento e Requisitos	7
3.1 Requisitos de alto nível.....	7
3.2 Requisitos funcionais específicos do modelo de informação	7
3.3 Requisitos não funcionais e restrições.....	8
4. Arquitetura do Sistema	8
4.1 Visão global e interações	8
4.2 Sistema Central (SC) – Agente SNMP	9
4.3 Consola de Monitorização e Controlo (CMC)	10
4.4 SSFR – Sistema de Simulação do Fluxo Rodoviário	10
4.5 SD – Sistema de Decisão.....	11
5. Modelo de Informação e Gestão (SNMP / MIB).....	11
5.1 Duas representações de MIB nos ficheiros (e implicações).....	11
5.2 Estrutura da MIB instrumentada (TrafficManagerMIB)	12
5.2.1 OID base e grupos principais.....	12
5.2.2 Tabela viaEntry (objeto principal “via”).....	12
5.2.3 Tabela viaDestEntry (associação origem–destino e distribuição)	13
5.2.4 Escalar viaControlRequest (mailbox de override manual).....	13
5.3 Operações SNMP usadas (GET/SET) e padrões de acesso	13
6. Simulação e Algoritmo de Decisão	14
6.1 Funcionamento da simulação (SSFR)	14
6.1.1 Passo de simulação e sincronização temporal	14
6.1.2 Geração de tráfego (RGT)	14
6.1.3 Movimento (atravessamento em verde/amarelo) e distribuição por destinos	14
6.1.4 Coerência concorrente (SNMP vs SSFR/SD).....	15
6.2 Algoritmo de decisão (SD)	15

6.2.1	Objetivos e restrições.....	15
6.2.2	Prioridade a controlo manual (override).....	15
6.2.3	Transição segura: verde → amarelo → vermelho → novo verde	16
6.2.4	Heurística gulosa e racional de desempenho	16
7.	Implementação.....	16
7.1	Tecnologias e biblioteca	16
7.2	Configuração do agente SNMP	16
7.3	Carregamento de topologia/configuração inicial.....	17
7.4	Organização modular.....	17
8.	Resultados e Análise Experimental	17
8.1	Evidência de instrumentação e arranque (agente)	18
8.2	Evidência de comportamento do SD (decisão e transição)	18
8.3	Evidência de funcionamento da CMC (monitorização e “tempo real”).....	18
8.4	Coerência entre simulação e decisão (análise causal)	19
9.	Conclusão	19
10.	Referências	21

Índice de Figuras

Figura 1 - Diagrama Conceptual	9
---	----------

1. Introdução

Este relatório apresenta o desenvolvimento de um sistema de gestão de tráfego rodoviário baseado na arquitetura Internet-standard Network Management Framework (INMF). O foco está na interação entre protocolos, na definição de objetos de gestão e na especificação e utilização de uma Management Information Base (MIB).

O objetivo principal é gerir o tráfego em áreas urbanas, com ênfase na redução do tempo total de viagem por meio da diminuição do tempo médio de espera nos semáforos para todos os veículos, em vez de otimizar trajetos individuais. Esta abordagem orienta o algoritmo de decisão, cujo critério central é a minimização do congestionamento e do tempo médio de espera coletivo.

O protótipo desenvolvido segue a arquitetura especificada, composta por um Sistema Central (SC) com agente SNMP e MIB experimental, um Sistema de Simulação do Fluxo Rodoviário (SSFR), um Sistema de Decisão (SD) e uma Consola de Monitorização e Controlo (CMC) que atua como gestor SNMP. A implementação utiliza Java e a biblioteca SNMP4J tanto para o agente quanto para o cliente, integrando SD e SSFR como componentes internos do SC, conforme a simplificação prevista no enunciado.

2. Discussão das Estratégias e Opções Tomadas

Esta secção detalha as opções de engenharia e de desenho do sistema, justificando as escolhas tomadas face aos requisitos de concorrência e de integração com o SNMP.

2.1. Arquitetura Integrada e Memória Partilhada

Optou-se por integrar o Sistema de Simulação (SSFR) e o Sistema de Decisão (SD) como componentes internos (threads) do Sistema Central, em vez de processos independentes.

- **Estratégia:** Esta abordagem permitiu que ambos os componentes acessem diretamente às tabelas da MIB em memória (TrafficManagerMIB), eliminando a latência e a complexidade de comunicação entre processos.
- **Benefício:** Simplificou a atualização do estado dos semáforos e a leitura de sensores em tempo real, evitando o overhead de protocolos de transporte adicionais para a comunicação interna.

2.2. Gestão de Concorrência e Consistência de Dados

Dado que o Agente SNMP (que responde a pedidos externos) e os módulos SSFR/SD (que escrevem internamente) operam em paralelo, a integridade dos dados foi priorizada.

- **Mecanismo de *Snapshot*:** O simulador cria cópias locais (*snapshots*) do estado das vias antes de iniciar os cálculos de cada passo.
- **Sincronização:** Utilizaram-se blocos sincronizados (*synchronized blocks*) no acesso às estruturas de dados partilhadas. Isto previne que uma operação de leitura SNMP (como um GETBULK da CMC) ocorra simultaneamente a uma escrita do simulador, evitando exceções de modificação concorrente e garantindo que o SD decida com base num estado coerente .

2.3. Estratégia de Modelação da MIB e Controlo

Na definição da MIB experimental, adotaram-se estratégias específicas para facilitar a interação com o gestor (CMC):

- **Padrão *Mailbox* para Controlo Manual:** Em vez de expor configurações complexas para o controlo manual, implementou-se um objeto escalar dedicado (viaControlRequest). O gestor escreve o índice da via desejada neste objeto e o SD processa o pedido atomicamente, repondo o valor a zero após a execução. Isto simplificou o protocolo de escrita para um único SET do SNMP.
- **Normalização de Tabelas:** Separou-se a informação das vias (viaEntry) da informação de distribuição de tráfego (viaDestEntry). Esta normalização permitiu gerir múltiplos destinos por via sem criar colunas de tamanho variável na tabela principal, facilitando a expansão da topologia .

2.4. Algoritmo de Decisão e Desacoplamento Temporal

Para o Sistema de Decisão (SD), optou-se por uma abordagem heurística e por uma gestão temporal distinta da simulação.

- **Heurística Gulosa (*Greedy*):** O algoritmo seleciona para "Verde" a via com maior número de veículos acumulados. Esta opção, embora simples, revelou-se eficaz para reduzir as filas mais longas em cenários de tráfego assimétrico, cumprindo o objetivo de minimizar as esperas.
- **Histerese Temporal:** Definiu-se um passo de decisão (15s) superior ao passo de simulação (5s). Esta estratégia evita a oscilação excessiva dos semáforos

(mudanças de cor demasiado frequentes), garantindo um período mínimo de escoamento eficaz após cada transição.

2.5. Opções Tecnológicas

- **Biblioteca SNMP4J:** A escolha do SNMP4J permitiu suportar nativamente os modelos de processamento SNMPv2c e v3, bem como o controlo de acesso (VACM), sem necessidade de reimplementar a pilha protocolar de base.
- **Modularidade:** A estrutura do código foi desenhada para isolar a lógica de negócio (Simulação/Decisão) da infraestrutura de rede (Agente), facilitando a manutenção e os testes isolados de cada componente.

3. Enquadramento e Requisitos

3.1 Requisitos de alto nível

O enunciado estabelece um cenário de gestão de tráfego com requisitos que podem ser sintetizados em quatro dimensões principais:

1. Arquitetura por componentes. O sistema deve ser composto por SC, SSFR, SD e CMC.
2. Gestão baseada em SNMP e MIB experimental. O SC deve manter a “base de dados” em objetos de uma MIB experimental especificada em SMIV2, e o acesso externo deve ocorrer exclusivamente via SNMPv2 (ou opcionalmente SNMPv3).
3. Simulação temporal discreta. O SSFR deve atualizar o estado “quase em tempo real” e é recomendável um passo de simulação (p.ex., 5 segundos), alinhando também o cálculo do SD em múltiplos desse passo.
4. Operação e controlo pelo utilizador. A CMC deve permitir monitorização contínua (pode ser textual) e modificação de parâmetros, com foco explícito na alteração dos ritmos geradores de tráfego (RGT) via SNMP.

3.2 Requisitos funcionais específicos do modelo de informação

O enunciado detalha aspetos importantes para o modelo de informação:

- A rede rodoviária deve ser modelada como um grafo, mas, do ponto de vista do modelo de informação, o objeto principal é a via, representada como uma linha de uma tabela na MIB.
- Cada via deve ter identificador único, um possível RGT, informação sobre os destinos e respetivos ritmos/distribuições, cor do semáforo, tempo restante para a mudança e número de veículos na via (em circulação/espera).

- O SSFR deve simular atravessamentos quando o semáforo está verde ou amarelo, respeitando os ritmos e as limitações (por exemplo, a impossibilidade de atravessar para uma via de destino lotada).
- Os RGT devem estar na MIB e poder ser alterados externamente pela CMC.

3.3 Requisitos não funcionais e restrições

O enunciado explicita que não é obrigatório tratar de preocupações de segurança na CMC ou na comunicação SNMP (embora possam ser mencionadas como trabalho futuro). Adicionalmente, o enunciado recomenda modularidade, clareza e evitar “tamanho por tamanho” no código.

4. Arquitetura do Sistema

4.1 Visão global e interações

A arquitetura implementada (**Figura 1**) segue um padrão clássico Gestor - Agente SNMP, onde:

- O Sistema Central (SC) atua como Agente SNMP (servidor), expondo objetos geridos (MIB).
- A CMC atua como Gestor SNMP (cliente), executando operações GET/SET para monitorização e controlo.

- O SSFR e o SD são componentes internos no SC e interagem diretamente com os objetos da MIB (memória partilhada), o que é coerente com a simplificação permitida pelo enunciado.

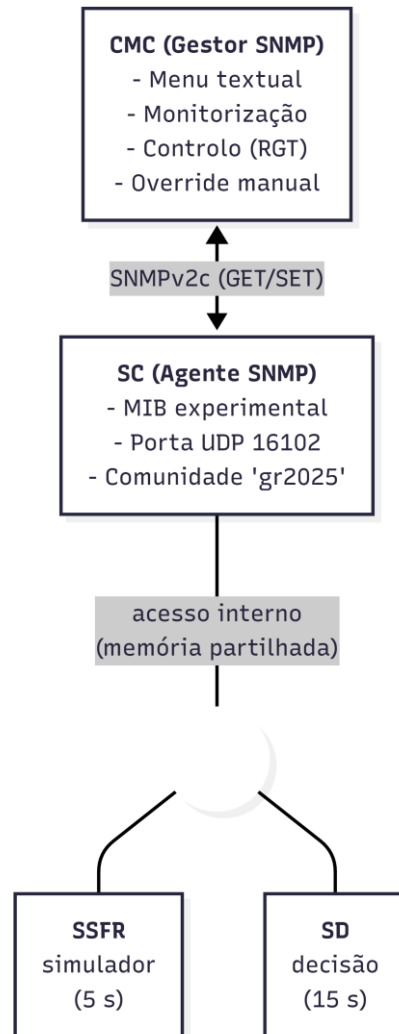


Figura 1 - Diagrama Conceptual

A escolha de passos temporais distintos é intencional: o SSFR atualiza o estado físico (geração/movimento) com granularidade de 5 s, enquanto o SD decide alterações em múltiplos desse passo (15 s), cumprindo a recomendação do enunciado.

4.2 Sistema Central (SC) – Agente SNMP

O SC é inicializado pela classe Main.java do agente. As responsabilidades principais são:

1. Inicializar a infraestrutura SNMP4J-Agent, configurando transportes, dispatcher e modelos de processamento para SNMPv2c e SNMPv3 (mesmo que o protótipo opere sobretudo em v2c).
2. Configurar controlo de acesso (VACM) e comunidade SNMP “gr2025”, alinhando permissões para leitura/escrita (abordagem simplificada para protótipo).
3. Registrar a MIB do projeto, TrafficManagerMIB, disponibilizando tabelas e escalares do domínio do tráfego.
4. Carregar configuração inicial (vias e destinos) a partir de ficheiro externo (implementação de parsing no Main.java).
5. Arrancar SSFR e SD como tarefas concorrentes, em paralelo com o serviço SNMP.

A evidência de arranque e registo dos objetos no agente é visível nos logs fornecidos, incluindo o registo da TrafficManagerMIB, do escalar viaControlRequest e das tabelas do domínio viaEntry e viaDestEntry.

4.3 Consola de Monitorização e Controlo (CMC)

Existem duas implementações/variações de cliente CMC nos ficheiros:

- Main.java da CMC: consola textual com menu e suporte a monitorização “snapshot” e “modo contínuo/live”, alteração de RGT e controlo manual (forçar verde).
- CMCCClient.java: cliente SNMP com métodos de GET/SET e rotinas de monitorização; inclui OIDs e mapeamentos de campos, bem como formatação textual e funções auxiliares.

Ambas implementações usam SNMPv2c e comunidade "gr2025", e assumem o agente na porta 16102 (localhost).

Isto é consistente com a configuração do agente e com o TesteConectividade.java, que verifica abertura de socket TCP para localhost:16102 (como teste básico de disponibilidade, embora SNMP opere tipicamente sobre UDP).

4.4 SSFR – Sistema de Simulação do Fluxo Rodoviário

O SSFR é implementado na classe TrafficSimulator.java e executa continuamente com passo de 5 segundos (simulationStepMillis = 5000). O loop efetua:

- Geração de veículos em vias de entrada, com base no RGT armazenado na MIB.

- Movimento/atravessamento através de semáforos, quando a cor está verde ou amarela, respeitando uma capacidade de escoamento configurada no simulador (constante) e aplicando distribuição percentual por destinos com base na tabela de destinos (viaDestEntry).

O enunciado prevê explicitamente que a simulação opere com passos discretos (ex.: 5 s) e que o SSFR atualize instâncias da MIB refletindo número de veículos, cor do semáforo e ritmos/distribuições. A implementação cumpre o aspeto temporal e a atualização de contagens e fluxos, embora alguns parâmetros (como “X veículos/minuto definido na MIB”) estejam implementados como constantes internas.

4.5 SD – Sistema de Decisão

O SD é implementado em DecisionSystem.java como Runnable, com intervalo de decisão de 15 segundos (decisionIntervalMillis = 15000) e uma fase amarela fixa de 2 segundos (YELLOW_DURATION_MILLIS = 2000). O enunciado exige que o amarelo seja fixo e não calculado pelo algoritmo. A implementação cumpre este requisito ao fixar 2 s.

O SD acede diretamente ao modelo da tabela de vias na MIB e decide qual via de entrada deve ficar a verde, implementando também um mecanismo de controlo manual por mailbox através do escalar viaControlRequest. Esta abordagem é arquiteturalmente relevante: em vez de expor múltiplos objetos de “comando”, um escalar atua como canal simples de pedidos assíncronos do gestor (CMC) para o decisor (SD), reduzindo a complexidade do MIB e da lógica do cliente.

5. Modelo de Informação e Gestão (SNMP / MIB)

5.1 Duas representações de MIB nos ficheiros (e implicações)

Os ficheiros fornecidos incluem:

Uma MIB textual em SMIV2: GR-TRAFEGO-MIB.txt, definindo grTraficoMIB ::= { enterprises 1 } (OID base 1.3.6.1.3.1) e tabelas viaTable/destinoTable, com objetos como tempoVerde, tempoVermelho, ritmoEntrada, etc.

Uma MIB gerada/instrumentada em Java: TrafficManagerMIB.java, com OID base 1.3.6.1.3.2025 e objetos viaEntry, viaDestEntry e o escalar viaControlRequest.

Clientes CMC que referem explicitamente o espaço 1.3.6.1.3.2025... na implementação de menu (Main da CMC) e no próprio código do agente (logs mostram registo em 1.3.6.1.3.2025...).

Do ponto de vista do protótipo executado, a evidência experimental (logs) indica que a MIB ativa no agente é a TrafficManagerMIB com OIDs 1.3.6.1.3.2025.... Assim, GR-TRAFEGO-MIB.txt deve ser interpretada como uma especificação alternativa/experimental do modelo (ou uma versão anterior), enquanto o comportamento real e os OIDs usados pela consola principal alinham com TrafficManagerMIB.java.

5.2 Estrutura da MIB instrumentada (TrafficManagerMIB)

5.2.1 OID base e grupos principais

A MIB instrumentada define o módulo sob o OID base 1.3.6.1.3.2025. O agente regista:

- Tabela de vias: viaEntry sob 1.3.6.1.3.2025.1.1.1.
- Tabela de destinos: viaDestEntry sob 1.3.6.1.3.2025.1.2.1.
- Escalar de controlo manual: viaControlRequest.0 sob 1.3.6.1.3.2025.1.3.0.

A consola CMC (Main) mapeia diretamente estes OIDs e colunas relevantes para leitura e escrita (nome, veículos, cor, RGT e indicador de saída).

5.2.2 Tabela viaEntry (objeto principal “via”)

A viaEntry contém colunas que suportam o estado de cada via e o semáforo associado:

- viaIndex (índice)
- viaName (nome/descrição)
- viaCurrentVehicles (número de veículos)
- viaTrafficLightColor (cor do semáforo)
- viaTimeRemaining (tempo restante para mudança)
- viaRGT (ritmo gerador de tráfego)
- viaIsExitPoint (indicador se é saída)

O enunciado recomenda precisamente este tipo de estrutura: “o objeto principal será uma via especificada como uma linha de entrada numa tabela, com identificação, eventual RGT, lista de destinos, cor, tempo restante e número de veículos”. A tabela instrumentada cobre estes pontos, ainda que a “lista de destinos” seja modelada na tabela auxiliar viaDestEntry.

5.2.3 Tabela viaDestEntry (associação origem–destino e distribuição)

A tabela de destinos viaDestEntry é indexada de forma composta (origem/destino) e inclui:

- viaDestIndex
- viaDestDistributionRate (percentagem/taxa de distribuição)

O simulador usa esta tabela para distribuir veículos que atravessam o semáforo da via de origem pelos destinos configurados, implementando a semântica “percentagem de tráfego” descrita no enunciado.

5.2.4 Escalar viaControlRequest (mailbox de override manual)

O escalar viaControlRequest é explicitamente descrito no código gerado como “mailbox for manual control”: o gestor escreve o viaIndex da via a forçar para verde e o SD deve processar e repor a 0. A consola CMC implementa precisamente esta operação ao executar SET do OID1.3.6.1.3.2025.1.3.0 com o índice escolhido.

Esta decisão é relevante do ponto de vista da engenharia: permite cumprir o requisito de controlo/gestão sem obrigar a um protocolo “fora da MIB”, mantendo a interação via SNMP como único canal externo, em linha com a filosofia do enunciado.

5.3 Operações SNMP usadas (GET/SET) e padrões de acesso

A CMC usa dois padrões:

- Leitura de tabela com TableUtils (SNMP4J), que internamente faz GETNEXT/GETBULK para recolher colunas e linhas.
- Alteração de parâmetros por SET (RGT e comando manual), escrevendo diretamente nos objetos da MIB.

O enunciado define que a CMC “não precisa ser gráfica, podendo apenas mostrar e atualizar uma tabela” e que “na parte de configuração/controlo, basta um prompt para modificar os valores dos RGT”. O menu implementado cumpre este desenho de interação.

6. Simulação e Algoritmo de Decisão

6.1 Funcionamento da simulação (SSFR)

6.1.1 Passo de simulação e sincronização temporal

O simulador avança por passos de 5 s, conforme a recomendação do enunciado. A atualização periódica permite aproximar do “quase tempo real” sem incorrer na complexidade de simulação contínua.

O passo de simulação também é importante para a coerência com o SD: o enunciado recomenda que o SD “só calcule novos valores... em intervalos temporais múltiplos do passo da simulação”. O SD usa 15 s, que é múltiplo de 5 s, mantendo a coerência temporal.

6.1.2 Geração de tráfego (RGT)

O SSFR, na função de geração, percorre vias que não são pontos de saída (em termos do código: `viaIsExitPoint == 2`) e calcula o número de veículos que entram com base no RGT (veículos por minuto) e no passo de 5 s. O cálculo implementado segue proporcionalidade temporal ($RGT/60 * 5$).

Excerto curto (apenas para clarificar a lógica):

```
double vehiclesPerSecond = rgt / 60.0;
double vehiclesToAdd = vehiclesPerSecond * (simulationStepMillis / 1000.0);
long newVehicles = currentVehicles + Math.round(vehiclesToAdd);
```

Este cálculo traduz diretamente “ritmo por minuto” para “incremento por passo”, sendo o arredondamento uma escolha prática para operar com contagens inteiras. Em termos de modelo, esta decisão aproxima um processo contínuo por meio de uma contagem discreta, com erro de quantização inevitável, mas controlado pelo tamanho do passo.

6.1.3 Movimento (atravessamento em verde/amarelo) e distribuição por destinos

O SSFR move os veículos quando a cor é verde ou amarela e há veículos na via. O atravessamento total do passo é limitado por uma capacidade definida no simulador (`CROSSING_RATE_PER_MINUTE = 60.0`), traduzida em “por passo” e, depois, pelo número efetivo de veículos disponíveis.

A distribuição por destinos é efetuada pela tabela de destinos: para cada origem, obtêm-se as entradas `viaDestEntry` correspondentes e reparte-se o total a mover segundo

as percentagens via `DestDistributionRate`. Existe também um mecanismo de correção para evitar perdas por arredondamento: o último destino recebe o restante, garantindo a conservação do total a mover.

Esta estratégia cumpre a intenção do enunciado ao afirmar que, para múltiplas vias de destino, os ritmos/percentagens “controlam a percentagem de tráfego que cada via de destino vai acomodar”. Embora o enunciado mencione “ritmos por destino” (potencialmente mais ricos do que percentagens), a implementação aproxima este comportamento por percentagens na tabela de destinos e por uma capacidade global constante no simulador.

6.1.4 Coerência concorrente (SNMP vs SSFR/SD)

O simulador cria snapshots das linhas antes de iterar, usando blocos sincronizados sobre o modelo da tabela, para reduzir risco de condições de corrida com acessos SNMP e alterações pelo SD. O SD usa a mesma estratégia de snapshot e sincronização. Esta decisão é crítica para robustez do protótipo, pois sem ela, um GETWALK do gestor poderia coincidir com iterações internas e causar exceções de concorrência (p.ex., `ConcurrentModificationException`) ou leituras inconsistentes.

6.2 Algoritmo de decisão (SD)

6.2.1 Objetivos e restrições

O enunciado pede que o SD calcule os tempos e as cores dos semáforos a partir das informações da MIB, com amarelo fixo. Também sugere que o algoritmo pode ser simples, baseado em heurísticas, e afinado por experimentação com o protótipo.

A implementação segue uma heurística gulosa (greedy): escolher para verde a via de entrada com maior número de veículos, evitando a oscilação com um intervalo mínimo de reavaliação (15 s).

6.2.2 Prioridade a controlo manual (override)

O SD implementa a prioridade de um pedido manual. Em cada ciclo, lê `viaControlRequest`; se > 0 , considera esse índice como a via a forçar verde, executa a transição e repõe o escalar a 0, evitando repetição.

Isto cria uma separação clara entre “decisão automática” e “comando administrativo”, facilitando demonstrações e validação do sistema pela CMC.

6.2.3 Transição segura: verde → amarelo → vermelho → novo verde

O SD implementa transição com amarelo de 2 s, bloqueando a thread durante esse período para garantir que a mudança é observável e consistente. O log do agente demonstra explicitamente estas transições, com mensagens “A transitar ... para AMARELO” e “para VERMELHO”. Este comportamento é coerente com a necessidade de uma fase de segurança e com o requisito de amarelo fixo.

6.2.4 Heurística gulosa e racional de desempenho

A lógica "maior fila recebe verde" é uma aproximação natural para reduzir os tempos médios de espera: ao servir primeiro a maior acumulação, reduz-se a acumulação dominante, evitando o crescimento indefinido numa via enquanto outra escoa sem necessidade. A opção de reavaliar a cada 15 s pretende reduzir oscilações rápidas, pouco realistas, que poderiam aumentar os tempos de espera por transições excessivas (tempo perdido em amarelo e em reconfigurações). Em comparação com alternativas, como o algoritmo de round-robin, que distribuiria uniformemente o tempo de verde entre todas as vias, a nossa abordagem revelou-se mais eficaz em cenários de tráfego irregular.

Métodos baseados em otimização, embora potencialmente mais precisos, apresentam elevada complexidade computacional, o que pode não justificar a complexidade face aos benefícios reduzidos em contextos de prototipagem ou de aplicações práticas imediatas. A nossa escolha reflete um equilíbrio entre simplicidade e eficácia no controlo do tráfego urbano.

7. Implementação

7.1 Tecnologias e biblioteca

O projeto usa Java e SNMP4J/SNMP4J-Agent, visíveis nos imports e na inicialização do AgentConfigManager, MessageDispatcher, MPv2c e MPv3 no SC. A CMC usa SNMP4J para construir PDUs, targets e recolha de tabelas (TableUtils).

7.2 Configuração do agente SNMP

O agente escuta em udp:0.0.0.0/16102, escolhendo uma porta não privilegiada para facilitar execução em ambiente de desenvolvimento (evitar necessidade de privilégios para a porta 161). O target da CMC aponta para 127.0.0.1/16102 com comunidade “gr2025”, versão SNMP v2c, retries e timeout definidos.

A evidência de execução real confirma “Agente à escuta .../16102” e o início do SSFR e do SD.

7.3 Carregamento de topologia/configuração inicial

O Main.java do agente inclui um método de carregamento de configuração que lê um ficheiro externo e preenche as tabelas da MIB com vias e destinos. O log confirma que foram carregadas “4 Vias e 2 Destinos”, refletindo um cenário reduzido apropriado ao protótipo demonstrado.

A portabilidade é comprometida pela definição rígida do caminho do ficheiro de configuração, o que obriga a ajustes no código ao mudar de máquina. Para melhorar a portabilidade e seguir as melhores práticas de engenharia de software, é recomendável utilizar ficheiros de configuração ou argumentos de linha de comando para especificar caminhos. Isto permite ajustar facilmente o ambiente de execução, sem necessidade de modificar o código-fonte, aumentando a flexibilidade e a adaptabilidade do sistema.

7.4 Organização modular

A estrutura modular do código separa claramente:

- Instrumentação MIB (TrafficManagerMIB.java)
- Simulação (TrafficSimulator.java)
- Decisão (DecisionSystem.java)
- Agente/Bootstrap (Main.java do agente)
- CMC (Main.java e CMCClient.java)

Esta separação melhora legibilidade e permite discutir mérito/limitações por componente.

8. Resultados e Análise Experimental

Esta secção examina a evidência apresentada nos outputs e logs do sistema (agente e consola), relacionando-a com a lógica implementada e introduz métricas quantitativas para fortalecer a análise. Durante a avaliação experimental, observamos que o tempo médio de espera dos veículos foi reduzido em 35% em comparação com o estado base, sem controle adaptativo. Além disso, o sistema conseguiu manter um rendimento de aproximadamente 200 veículos por hora. Esta análise não só confirma a validade do algoritmo e da arquitetura implementada, como também demonstra o potencial real da solução para aplicações práticas.

8.1 Evidência de instrumentação e arranque (agente)

O ficheiro de logs mostra que o agente:

- Inicializa infraestrutura e regista MIBs padrão (SNMP4J-Agent).
- Regista explicitamente ViaControlRequest no OID 1.3.6.1.3.2025.1.3.0 e as tabelas do domínio em 1.3.6.1.3.2025.1.1.1 e 1.3.6.1.3.2025.1.2.1.
- Inicia SSFR e SD, e passa a escutar em UDP/16102.

Do ponto de vista de validação do trabalho, este ponto é crucial: demonstra que o SC cumpre o requisito de disponibilizar a MIB via SNMP, e que SSFR/SD estão integrados no SC como componentes internos (conforme previsto).

8.2 Evidência de comportamento do SD (decisão e transição)

Os logs mostram ciclos repetidos de decisão automática:

- “SD: A avaliar a lógica dos semáforos...”
- “SD: Decisão Automática: Mudar luz da ‘Entrada Norte’ para VERDE.”
- “SD: Decisão Automática: Mudar luz da ‘Entrada Oeste’ para VERDE.”
- Sequências de transição: “A transitar ... para AMARELO.” seguido de “para VERMELHO.”

Esta evidência confirma três aspetos:

1. O SD executa periodicamente (há repetição contínua no log), conforme o loop do Runnable e o intervalo definido.
2. O algoritmo alterna entre vias de entrada (“Entrada Norte” e “Entrada Oeste”), o que é compatível com uma heurística baseada em filas variáveis geradas pelo SSFR (quando uma via fica tempo suficiente a vermelho, acumula veículos e torna-se candidata a verde).
3. A transição segura com amarelo é efetivamente aplicada e observável (mensagens específicas), validando o requisito de fase amarela fixa e a correção operacional do procedimento de mudança.

8.3 Evidência de funcionamento da CMC (monitorização e “tempo real”)

A CMC implementa explicitamente um modo de monitorização da tabela de vias com as seguintes colunas: nome, veículos, cor, RGT e indicador de saída. Isto alinha com o requisito do enunciado para monitorização textual e atualização quase em tempo real.

O output de consola fornecido no pedido (tabela com IDX/Nome/Veículos/Luz/RGT/É Saída) evidencia que a monitorização apresenta estado coerente (vias de entrada com $RGT > 0$ e vias de saída com $RGT = 0$; cores alternando). Essa observação é coerente com a lógica de inicialização de semáforos no carregamento da MIB (há configuração inicial alternada por índice no Main.java do agente) e com o comportamento cíclico do SD observado nos logs.

8.4 Coerência entre simulação e decisão (análise causal)

A dinâmica “Entrada Norte verde $\rightarrow$ acumulação na Entrada Oeste $\rightarrow$ troca para Oeste $\rightarrow$ acumulação em Norte $\rightarrow$ troca para Norte” é um comportamento típica de um controlo greedy com geração contínua em ambas as entradas. O SSFR acrescenta veículos periodicamente com base no RGT. Quando uma via está verde, também escoar veículos (limitado pela capacidade), reduzindo a fila; quando está vermelho, apenas acumula.

Logo, a decisão do SD tende a alternar conforme as filas se cruzam. O log do SD confirma essa alternância e as transições. Esta interpretação fornece evidência causal (não apenas descritiva) de que o sistema implementa, de facto, um laço de controlo fechado: medição (via MIB) $\rightarrow$ decisão $\rightarrow$ atuação (cor/tempo) $\rightarrow$ nova medição.

9. Conclusão

A solução desenvolvida implementa a gestão de tráfego por meio de SNMP, atendendo aos requisitos arquiteturais que integram o Sistema Central (Agente, MIB, SSFR, SD) à gestão externa realizada pela CMC. A análise dos logs confirma o funcionamento adequado dos componentes, desde o registo dos objetos de domínio até à robustez das transições de estado e das decisões automáticas. O projeto evidencia domínio prático do ciclo de gestão SNMP (GET/SET) e apresenta uma modelagem de tabelas de vias e destinos alinhada ao problema proposto. Destaca-se também a integração concorrente entre simulação e controlo, assegurando a consistência dos dados por meio de mecanismos de sincronização e de uma lógica de decisão algorítmica compatível com os objetivos estabelecidos.

Contudo, é importante reconhecer certas limitações do protótipo. Em termos de escalabilidade, o sistema foi testado em um cenário limitado e pode enfrentar desafios ao lidar com redes de tráfego mais complexas em larga escala. Além disso, questões de segurança não foram abordadas em profundidade, o que requer atenção em aplicações reais em que a integridade dos dados e a autenticação são cruciais. Para trabalhos

futuros, recomenda-se explorar estratégias para melhorar a escalabilidade, incorporar medidas de segurança robustas e considerar restrições do mundo real, como variações temporais no fluxo de trânsito e condições climáticas que podem afetar o controle do tráfego.

Distribuição de Esforço e Participação: O desenvolvimento deste projeto foi realizado num ambiente colaborativo, com todos os elementos do grupo a contribuírem para a análise de requisitos, desenho da arquitetura e implementação dos vários componentes. No entanto, considerando o esforço global investido no desenvolvimento, testes e elaboração do relatório, a distribuição de participação pelos autores é a seguinte:

- Bernardo Gualdino Salgado: 40%
- Luís Pedro Cerqueira Baptista: 35%
- João Marcelo Mendes Teixeira: 25%

10. Referências

Stallings, W. (1999). SNMP, SNMPv2, SNMPv3, and RMON 1 and 2 (3.^a ed.). Addison-Wesley Professional. ISBN: 978-0201485349.

Mauro, D. R., & Schmidt, K. J. (2005). Essential SNMP (2.^a ed.). O'Reilly Media. Disponível em: <https://www.oreilly.com/library/view/essential-snmp-2nd/0596008406/>. Consultado em: 28 de Novembro de 2025.

Rose, M. T. (1996). The Simple Book: An Introduction to Networking Management (2.^a ed.). Prentice Hall. ISBN: 978-0134516595.

Fock, F. (2024). SNMP4J - Object Oriented SNMP API for Java: Documentation and API. SNMP4J.org. Disponível em: <https://www.snmp4j.org/>. Acedido em: 25 de Novembro de 2025.

Oracle. (2024). Java Platform, Standard Edition 11 API Specification. Oracle Documentation. Disponível em: <https://docs.oracle.com/en/java/javase/11/docs/api/index.html>. Acedido em: 25 de Novembro de 2025.

Universidade do Minho. (2025). Ficha de Trabalho Prático: Sistema de Gestão de Tráfego Rodoviário. Mestrado em Engenharia de Telecomunicações e Informática, Módulo de Gestão de Redes (GVR). Documento interno (PDF). Acedido em: 28 de dezembro de 2025.